CYBER FORENSICS CAPSTONE | MOBILE FORENSICS | FINAL REVIEW REPORT

Android Deleted Data Recovery and Evidence Reconstruction Framework

Recovering deleted and superseded records from Android SQLite databases, carving raw evidence, correlating it across sources and scoring every finding, with a desktop application, a dashboard and a tamper-evident chain of custody

Project: Android Deleted Data Recovery and Evidence Reconstruction Framework

Author: Harsha Mithran

Programme: BTech, Cyber Forensics Capstone

Date: October 2026

Tools: Python 3.12, SQLite (read directly at page level), FastAPI, Qt (PySide6), pydantic, Docker, GitHub Actions

Version: 0.3.0 | **Repository:** github.com/MithranHarsha/Android-Forensic-Analysis-Toolkit

Contents

1. Solution Required
2. Solution Proposed
3. Algorithm
4. Outcome
5. Conclusion
6. Inference
7. Reference
Appendix A. Running the tool and the demo
Appendix B. Case directory and database schema

Summary. Most of what a phone user deletes is not erased: SQLite marks the space free and the old bytes often survive in freed pages, freeblocks, the write-ahead log and rollback journals. This project reads an Android acquisition without ever modifying it, proves its integrity, extracts the live data, and then recovers deleted and superseded records by parsing the database file format itself rather than asking SQLite. It carves loose patterns, correlates evidence across independent databases, and gives every recovered item a deterministic confidence level with the reasons written out. It was built from a synthetic-data prototype into a tool that has been run on a real public Android 9 acquisition, with a command line, a native desktop application, a dashboard, a hardened multi-user server, signed reports and a hash-chained audit log.

1. Solution Required

1.1 Problem statement

Recover deleted and overwritten data (messages, call records, contacts and browser history) from Android devices and present it as reliable, attributable evidence, while keeping the original acquisition unmodified.

Item	Specification
Objective	Recover deleted and superseded records from Android SQLite databases and reconstruct them into typed, explainable evidence with provenance.
Approach	Acquire and hash the evidence; discover databases; extract live artifacts; parse SQLite pages, freelists, the WAL and journals directly; carve raw patterns; correlate across sources; score confidence; validate; report.
Tools / technology	Python; a purpose-built SQLite page parser; read-only ext2/3/4 and f2fs readers; FastAPI; Qt.
Expected output	A case containing active and recovered artifacts, carved candidates, correlations, confidence assessments, a timeline, validation results and a signed forensic report.
Deliverable structure	Acquisition, Analysis, Validation, Report.

1.2 Why a solution is needed

Phones are the primary evidence source in many investigations, and users routinely delete the messages and call records that matter. Ordinary extraction tools show only what the application still lists. Deleted rows are not removed from the file: SQLite only marks their space reusable, so remnants can persist in freed pages and in the write-ahead log. Recovering them by hand means reading binary file formats, and doing so carelessly can alter the evidence. A forensic tool must therefore recover aggressively and still never alter the source, never overstate how sure it is, and always show where each finding came from.

1.3 Requirements

Functional requirements

- Ingest directories, ZIP and TAR archives, raw, Android-sparse and E01 disk images (ext2/3/4 and f2fs) without modifying them.
- Discover every SQLite database with its WAL, SHM and journal, and flag corrupt or encrypted ones instead of failing.
- Extract live SMS, calls, contacts and browser history, and catalogue every other table generically.
- Recover deleted and superseded records from the main file, the WAL and rollback journals, and carve loose patterns outside structured records.
- Correlate findings across independent databases and assign each a confidence level with written reasons.
- Validate the result, build a unified timeline and produce HTML, JSON, CSV and PDF reports.
- Offer a command line, a desktop application and a dashboard for review.

Non-functional requirements

- Forensic soundness: read-only access, SHA-256 hashing, a hash-chained audit log, signed reports.
- Safety: evidence is hostile input; malformed files must produce a declared error, never a crash, hang or escape.
- Explainability: no machine learning in any decision; every score lists its factors.
- Honesty: a carved candidate is never presented as confirmed evidence, and the absence of a result is never presented as proof of absence.
- Practical speed on phone-sized evidence (minutes, not hours).

1.4 The gap: a synthetic prototype versus real devices

The first release was validated only on a synthetic lab with exact ground truth. Real acquisitions differ in ways that decide whether such a tool is useful, so the required solution went beyond the first release.

What real evidence has	Problem in the first release	Change made
Disk images, not folders	Only directories and archives were read.	Raw, Android-sparse and E01 readers; hostile-input-hardened ext2/3/4 and f2fs parsers.
Hundreds of databases	Recovery did not finish on 605 real databases in 35 minutes.	Record header parsed once per byte offset instead of once per table: 7 to 10 times faster, results proven identical.
Hash-named cache files	The encryption detector flagged 7 directories of a decrypted image.	Restricted to block-aligned ciphertext lengths under an encryption-policy root.
Google Messages	Its private database was not parsed, so 3 reference messages were missed.	New parser: 184 messages on the real image, direction agrees on 30 of 30 matched messages.
Duplicated WAL history	Quadratic duplicate merging stalled large cases.	Indexed merge, proved equivalent to the original on random inputs.
Chain of custody	Only an in-database log.	External hash-chained audit log, Ed25519 report signing, build stamp.
Multiple analysts, shared lab	Single-user dashboard.	Authenticated server with roles, sandboxed jobs, SSO via a trusted proxy, rate limiting.
Non-technical reviewers	Command line and browser dashboard only.	Native desktop application with case library and drag-and-drop analysis.

> **Scope note.** Real validation used one public acquisition: the Digital Corpora Android 9 (Pixel 3) image, a logical extraction of a decrypted `/data` partition. No encrypted image, Samsung or Xiaomi data, or newer Android versions were available. Those remain unvalidated and are stated as such in Section 5.

2. Solution Proposed

2.1 Overview and architecture

A single pipeline serves the command line, the desktop application, the dashboard and the server. Each stage is a separate module and writes to one case database, so any stage can be tested or re-run on its own.

Layer	Responsibility
Acquisition	Hashing and safe ingestion of directories, ZIP/TAR, raw/sparse/E01 images; ext2/3/4 and f2fs readers; archive-attack and path-traversal defences; encryption detection.
Discovery	Filesystem classification, Android path analysis, SQLite database and sidecar (WAL, SHM, journal) discovery, header and version inspection.
Extraction	Schema-probing artifact parsers (SMS, calls, contacts, browser, Google Messages) plus a generic parser for every other table; plugin API for more.
Recovery	Page-level SQLite engine: freelist pages, freeblocks, unallocated space, WAL frames and rollback-journal pages; raw carving of phone numbers, URLs, e-mail addresses, timestamps, SMS-like text and loose record shapes.
Analysis	Correlation evidence graph, deterministic confidence scorer, validation checks, unified timeline.
Storage	One SQLite case database with versioned migrations, plus an external JSONL audit log and a signed evidence manifest.
Interface	Command line, native desktop application, read-only dashboard, authenticated multi-case server, HTML/JSON/CSV/PDF reports.

Data flow. Evidence, then SHA-256 recorded, then filesystem and database discovery, then active extraction, then recovery and carving, then correlation, then confidence scoring and reconstruction, then validation, then timeline, then report and signature.

2.2 Methodology: Acquisition, Analysis, Validation, Report

Acquisition. Every source is hashed with SHA-256 before it is read, and the hash is stored in the case manifest and the audit log. The original is opened read-only; working copies are made where needed and are write-protected. SQLite itself never touches the evidence: it works on a scratch copy and runs under a read-only authorizer with `trusted_schema` off, a heap cap and a per-statement deadline. Malformed archives, symlink escapes and oversized sidecars are rejected with a declared error.

Analysis. The stages below run in order. A damaged database never stops the run; it is flagged and the pipeline continues.

Stage	What it does
Scan	Classifies every file, detects Android roots, users and packages, finds databases with their WAL/SHM/journal, and flags encrypted-looking files.
Extract	Opens each database read-only and extracts typed artifacts with full provenance (database, table, row id, page, byte offset, raw bytes).

Recover	Parses freed and superseded cells from main-file pages, WAL frames and journal pages, fits them to the table schema and decodes every column it can prove.
Carve	Matches phone numbers, URLs, e-mail addresses, timestamps and SMS-like text in unstructured bytes, with context checks.
Correlate	Builds an evidence graph (people, numbers, messages, calls, contacts, URLs, timestamps) and finds independent corroboration.
Reconstruct	Turns recovered rows into typed artifacts through the same parsers used for live rows, and assigns confidence.
Validate / Timeline	Re-hashes evidence, checks SQLite integrity, schema, datatypes, timestamps, duplicates and provenance; merges everything into one chronological timeline.

Validation. Hashes are re-verified on demand against the manifest. The audit log is a hash chain, so a changed or deleted entry is detected. Recovery was checked against physical ground truth, compared with three other tools and run on a real acquisition (Section 4).

Report. HTML, JSON, CSV and PDF reports carry the provenance of every item, the confidence explanations and a recoverability disclaimer; reports can be signed with Ed25519 (GPG optional). HTML tables are capped for readability with summaries that cover every row, while JSON and CSV are always complete.

2.3 The desktop application and dashboard

The desktop application is a native window that reads finished cases read-only and runs the analysis as a separate child process, so it adds no new analysis path: what it shows is exactly what the command line produced. A case library sits on the left. Evidence is dropped onto the window, or chosen with a file or folder picker, and a progress view follows. Results open in tabs for the overview, active data, recovered (deleted) records, carved candidates, timeline, validation and chain of custody, each sortable and filterable.

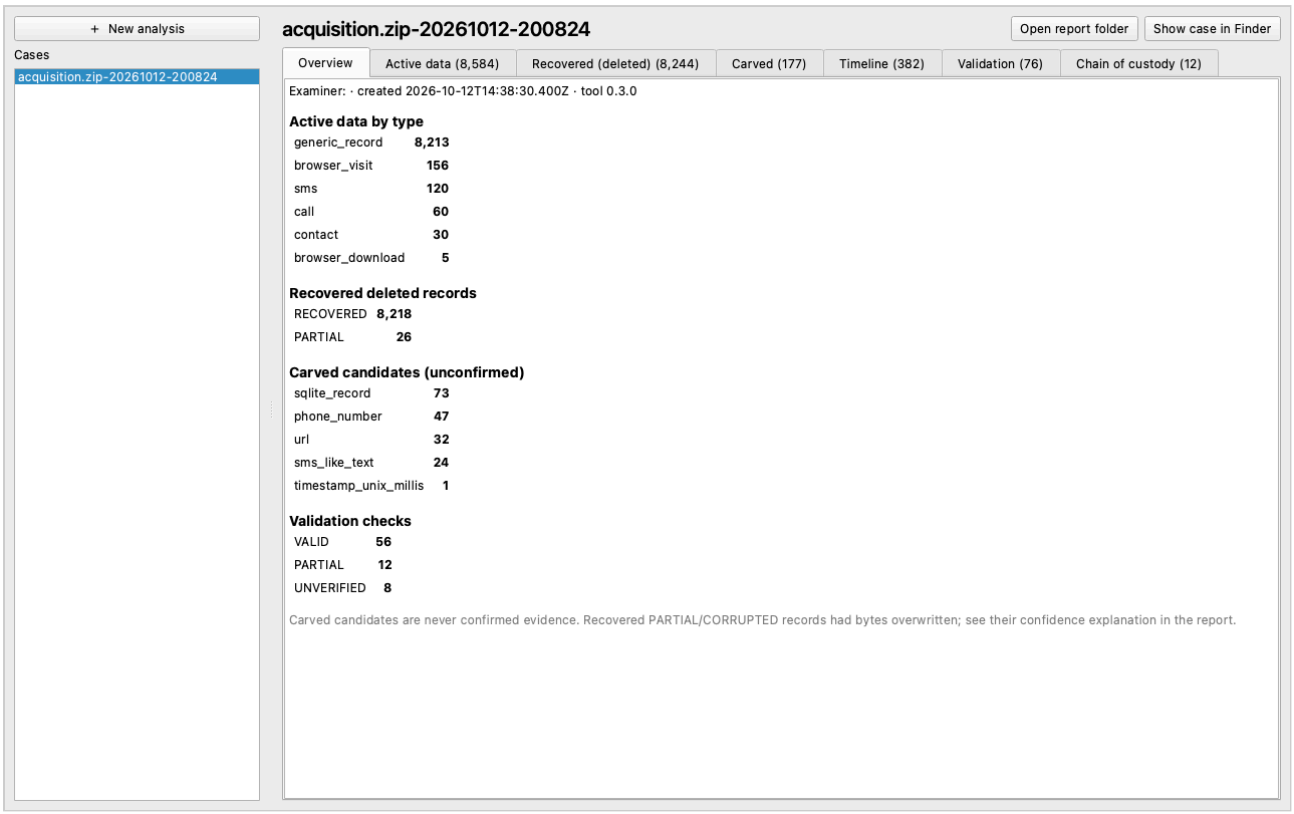


Figure 1. Desktop application: case library and overview of a synthetic case (8,244 recovered records, 177 carved candidates, 76 validation checks).

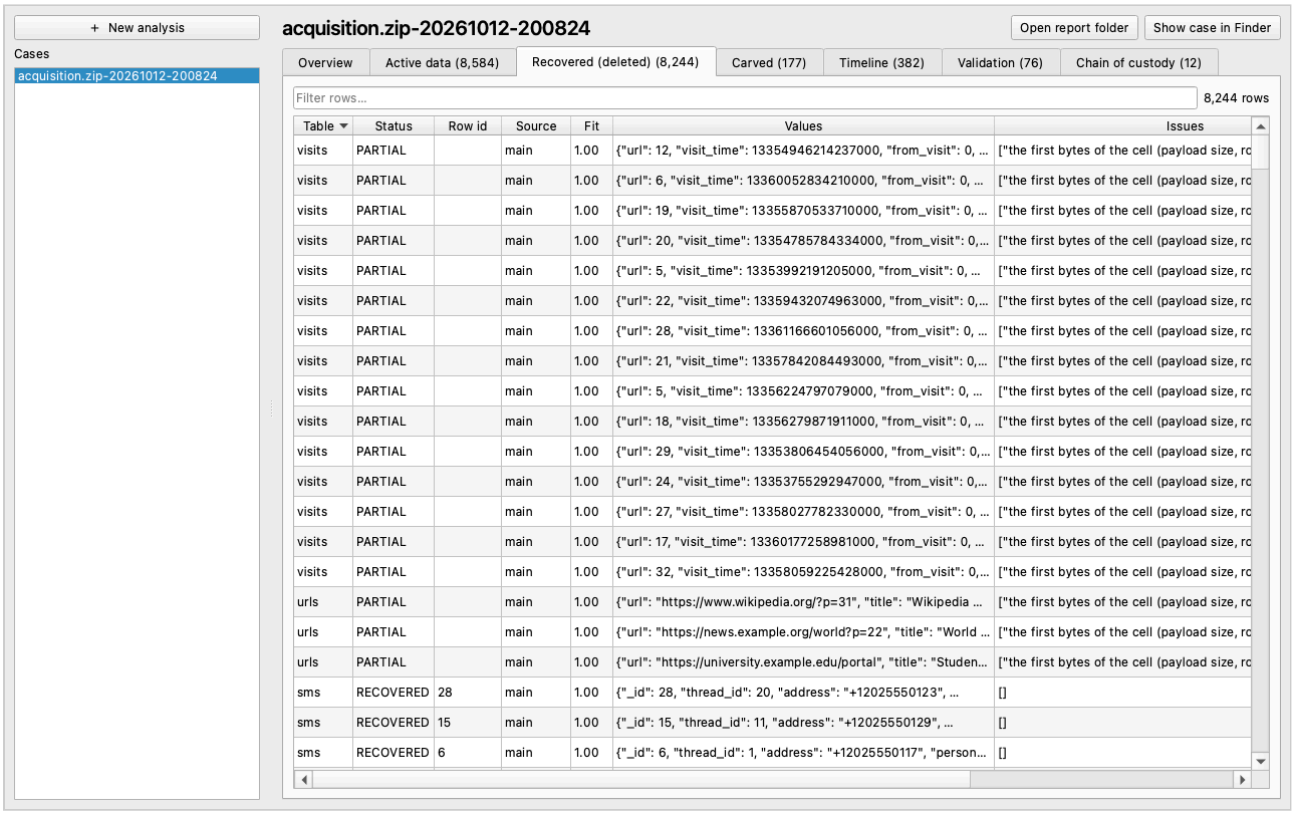


Figure 2. Recovered (deleted) records, sortable and filterable. PARTIAL rows lost their row id to a freeblock header.

The browser dashboard (`android-forensics dashboard`) is read-only. The multi-case server (`android-forensics serve`) adds token authentication with viewer, analyst and admin roles, streamed uploads,

sandboxed analysis jobs with resource limits, an audit log and optional single sign-on through a trusted proxy.

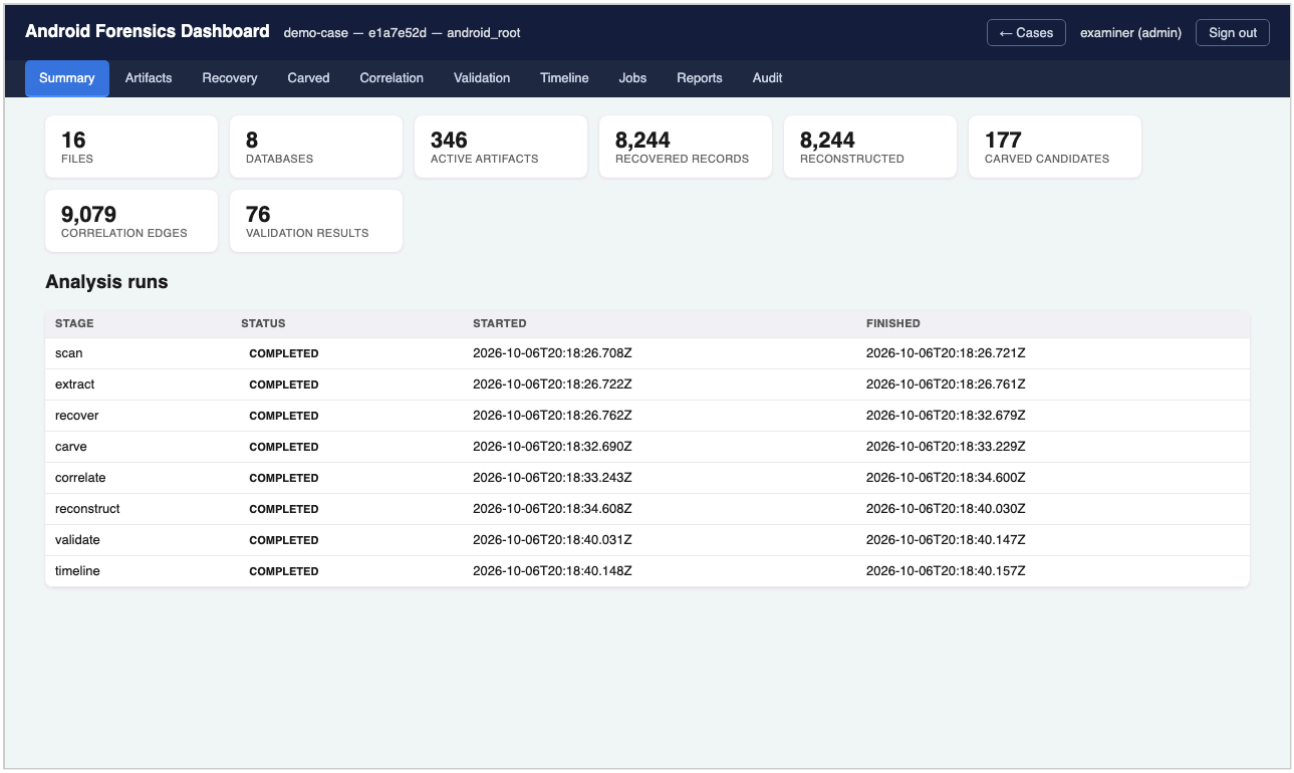


Figure 3. Dashboard summary: active artifacts, recovery results and validation.

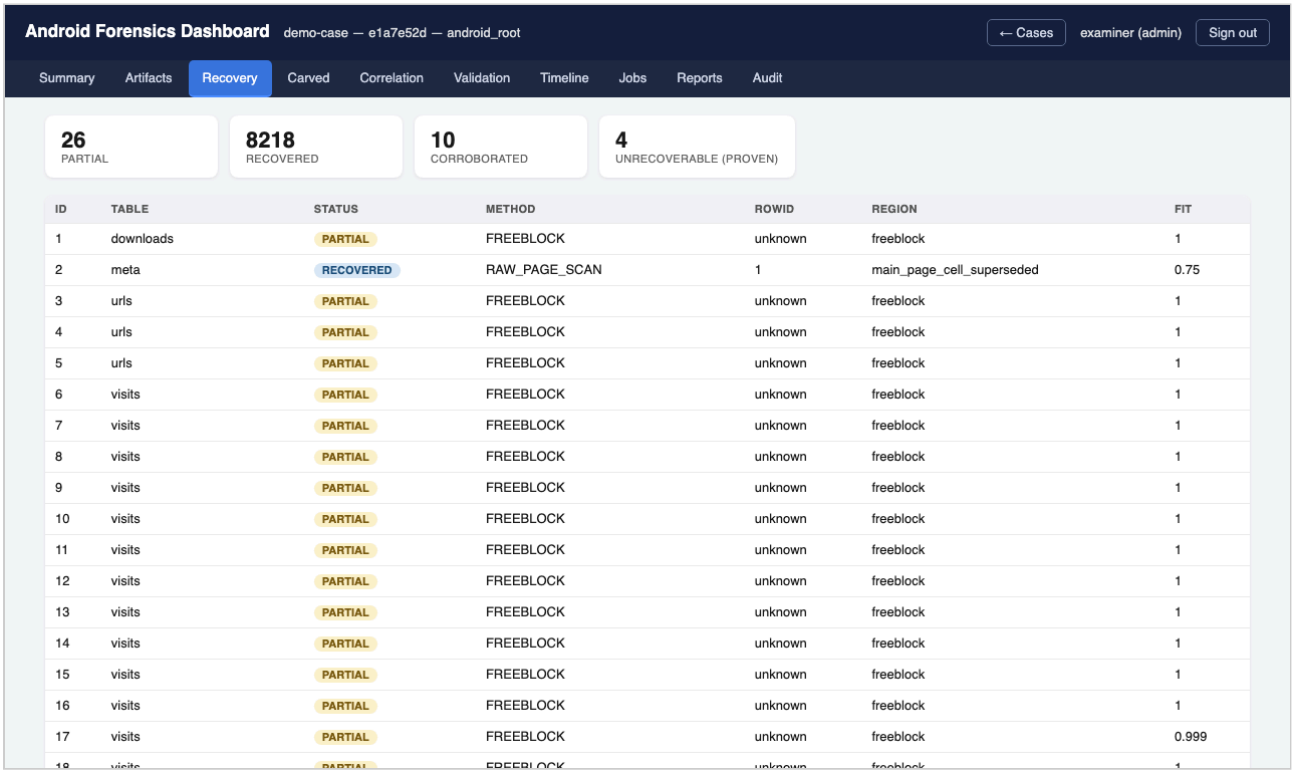


Figure 4. Dashboard recovery view with status, source and confidence.

2.4 Security design

The tool opens other people's data, so every input is treated as hostile.

- Evidence is read-only; SQLite runs under a read-only authorizer, `trusted_schema=OFF`, a heap cap and a per-statement deadline; XML is parsed with defusedxml.
- The ext4 and f2fs readers validate every length, offset and checksum (for f2fs, the checkpoint CRC and node-footer cross-checks) and have been fuzzed with mutated images.
- Control characters from evidence are stripped before reaching the terminal; reports escape everything; the web interface uses a strict Content-Security-Policy with no inline script.
- The server uses hashed tokens, role checks, rate limiting with lockout, host-header checks and a process-group sandbox with resource limits for jobs.
- Reports can be signed (Ed25519); the audit log and in-database custody log are hash-chained; each report carries a tool stamp.

2.5 Technology

Component	Choice and reason
Language	Python 3.11 or newer (developed on 3.12); typed throughout and checked with mypy on every module.
SQLite parsing	Purpose-built page, record, DDL, WAL, journal and freelist parsers so recovery does not depend on SQLite and can read what SQLite refuses to open.
Models	pydantic models for every artifact, finding and assessment.
Case storage	One SQLite database per case with versioned migrations; no server to install.
Interfaces	Typer and Rich (command line), FastAPI (dashboard and server), Qt via PySide6 (desktop application), Jinja2 and fpdf2 (reports).
Packaging	Hash-pinned lock files, a non-root read-only Docker image, a PyInstaller build, a macOS app bundle builder, GitHub Actions for lint, tests, fuzzing, container and binaries.

3. Algorithm

3.1 Overall pipeline

```
ANALYZE(evidence, case):
  for each source: record SHA-256 BEFORE reading                      // preservation
  discover files, databases and sidecars (WAL / SHM / journal)
  for each database (in parallel):                                     // errors
isolated per database
      extract live artifacts through schema-probing parsers
      candidates <- RECOVER(main file pages, WAL frames, journal pages)
      carved     <- CARVE(unstructured bytes and free space)
  correlate(active artifacts + recovered drafts)  // pass 1: what supports each record?
  reconstruct recovered rows with their corroboration; score confidence
  correlate(stored artifacts)                    // pass 2: persist final edges
  validate; build timeline; write reports; sign; audit-log every step
```

Recovery and carving are pure functions of one file's bytes, so they run in parallel (`--workers`). Results are written by a single process in task order, so a parallel run produces exactly the same case content as a serial one, which a test asserts.

3.2 Where deleted data survives in SQLite

Location	Why data remains
Freelist pages	A page emptied by deletes is added to the freelist whole; its old cells stay until the page is reused.
Freeblocks and unallocated space inside a live page	A deleted cell becomes a freeblock whose first four bytes are overwritten with a link and size; the rest of the cell, usually the whole record body, remains.
Write-ahead log	Superseded page versions remain in earlier frames until a checkpoint and reuse overwrite them.
Rollback journal	Holds pre-transaction page images that can include rows later deleted.

3.3 Page-level record recovery

```
RECOVER(page, schema models):
  for each byte offset s in the free regions of the page:
      header <- parse cell header at s ONCE            // [payload size][rowid][record
header]
      for each table model with matching column count:
          if the serial types fit the model and header + body fill the payload exactly:
              decode every column; check each against its declared affinity
              keep as FULL candidate (rowid known) with a fit score
  for cells whose first bytes were destroyed by the freeblock header:
      parse the record header at the surviving offset -> PARTIAL candidate (rowid
unknown)
  drop candidates that overlap a stronger one; merge copies of the same record across
pages,
  WAL frames and journals, keeping the most complete and listing the other locations
```

A candidate is accepted only if its structure is exact: the header size plus the body size must equal the payload size and the cell must end where a cell can end. Columns the bytes do not determine are marked UNKNOWN, never guessed. Duplicate merging uses indexes (by content and by postings) and was proved equivalent to the original quadratic definition with an oracle test.

Each recovered row is labelled **RECOVERED** (whole cell survived), **PARTIAL** (some fields lost, typically the row id) or **CORRUPTED** (bytes partly overwritten). WAL frames carry their own status (committed, uncommitted, stale), which feeds the confidence.

3.4 Carving

For bytes that belong to no parsed record (free space, caches, unparsed files), shape matchers look for phone numbers, URLs, e-mail addresses, timestamps, SMS-like text and loose SQLite record shapes. A bare digit run is never a phone number, and a timestamp is not carved on its own. Overlapping hits of the same kind are merged keeping the earlier or longer. Every result has the status `CARVED CANDIDATE` and can never be rated HIGH. Timestamp-against-phone matching uses sorted spans and bisection, which removed an O(n×m) scan that had taken half the run time.

3.5 Correlation

A phone number is one node regardless of which database carried it: `+12025550101`, `202-555-0101` and `(202) 555-0101` all key to the trailing ten digits. Edges such as *phone SENT message*, *contact CONTACT_OF phone* and *recovered RECOVERED_FROM database* link artifacts. Corroboration must come from a **different database**: a hit in `x.db-wal` is not corroborated by a live row of `x.db`. A recovered record's timestamp must be structurally plausible for its matches to count. Correlation runs in two passes, first over active artifacts plus drafts of recovered records to learn what supports each, then again over the stored artifacts to persist the final edges.

3.6 Deterministic confidence scoring

Confidence answers how sure the tool is that the bytes are a genuine record of what they claim to be, not how important the record is. It is computed from weighted factors, with no model and no randomness.

Factor	Weight
Schema match	2

Valid record structure	2
Valid serial types	1
Field consistency	1.5
Page integrity	1
Source provenance (WAL frame status, journal checksum)	1.5
Timestamp validity (only if the record has one)	1
Raw-byte completeness	1.5
Cross-artifact corroboration	2
Same record found in several places	0.5

score = sum(weight × satisfied) / sum(weights); HIGH at 0.85 or more, MEDIUM at 0.60 or more, otherwise LOW. Caps that no score can exceed: no schema match or an inexact structure gives LOW; CORRUPTED gives LOW; PARTIAL gives at most MEDIUM unless two independent sources corroborate it; RECOVERED reaches HIGH only if its source is verified and its fields consistent; a carved candidate is at most MEDIUM, and LOW without corroboration.

3.7 Image readers

Raw, Android-sparse and E01 (EWF v1) readers present a seekable byte stream. Read-only ext2/3/4 and f2fs parsers walk it without mounting. The f2fs reader validates the checkpoint CRC (`crc32_le`), resolves the node address table with its bitmap and journal override, cross-checks node footers, and handles inline data and indirect nodes. Every length and offset from the image is bounds-checked, and the readers were fuzzed with mutated images and tested on images written by a real Linux kernel in containers.

3.8 Tamper-evident custody

```
entry_hash[i] = SHA-256( entry_hash[i-1] | timestamp | actor | action | target | detail )
entry_hash[0] is computed over the constant "GENESIS"
verify: recompute from the start; the first mismatch is the first altered or deleted
entry
```

The same construction is used for the in-database custody log and for the external audit log (`audit.log.jsonl`), so the two records can be compared with each other. Reports are signed with Ed25519; the signing key can be passphrase-protected.

4. Outcome

4.1 What was delivered

- An end-to-end pipeline from acquisition to signed report, usable from the command line, a native desktop application, a dashboard and a multi-user server.
- A page-level SQLite recovery engine covering main file, freelist, freeblocks, WAL and journals, plus carving, correlation and deterministic confidence scoring.
- Read-only disk-image support for raw, sparse and E01 images with ext2/3/4 and f2fs.
- Chain-of-custody features: external hash-chained audit log, Ed25519 report signing, tool stamp, evidence manifest.
- Docker image, locked dependencies, a PyInstaller build, GitHub Actions workflows and MkDocs documentation.
- About 22,000 lines of Python source, 1,664 automated tests and 94.1% line coverage (the optional desktop front end has its own tests and is excluded from the coverage gate).

4.2 Results on the synthetic acceptance run

The synthetic lab generates evidence with physically derived ground truth. The end-to-end run on its 16 files (8 SQLite databases, including one encrypted-looking, one truncated, one WAL-mode and one with a hot journal) produced:

Measure	Result
Active artifacts	8,584 in total, including 120 SMS, 60 calls, 30 contacts, 156 browser visits and 5 downloads
Deleted or superseded records found	8,244 (8,218 RECOVERED, 26 PARTIAL, 0 CORRUPTED)
Proven-empty unrecoverable regions	4
Carved candidates	177, of which 93 corroborated
Correlation edges	9,079
Validation checks	76: 56 valid, 12 partial, 0 invalid, 8 unverified
Timeline events	382

The large recovered count is expected: the call-log database is written row by row before a checkpoint, leaving many superseded page versions in its WAL, each a legitimate recoverable finding. The 26 PARTIAL rows are the genuinely deleted records whose row id was lost to a freeblock header. Every original file was re-hashed after the run against the manifest, with no mismatches.

4.3 Recovery accuracy against physical ground truth

The 28-condition deletion matrix varies position, record size, page size, overwrite level and journal mode. Ground truth is physical: the lab re-encodes each deleted cell exactly as SQLite wrote it and searches the files for those bytes, so "recoverable" never depends on any tool.

Measure	Result
Physically recoverable rows	511
Recovered correctly	507 (recall 99.2%)
False positives	0
RECOVERED / PARTIAL / missed	146 / 361 / 4

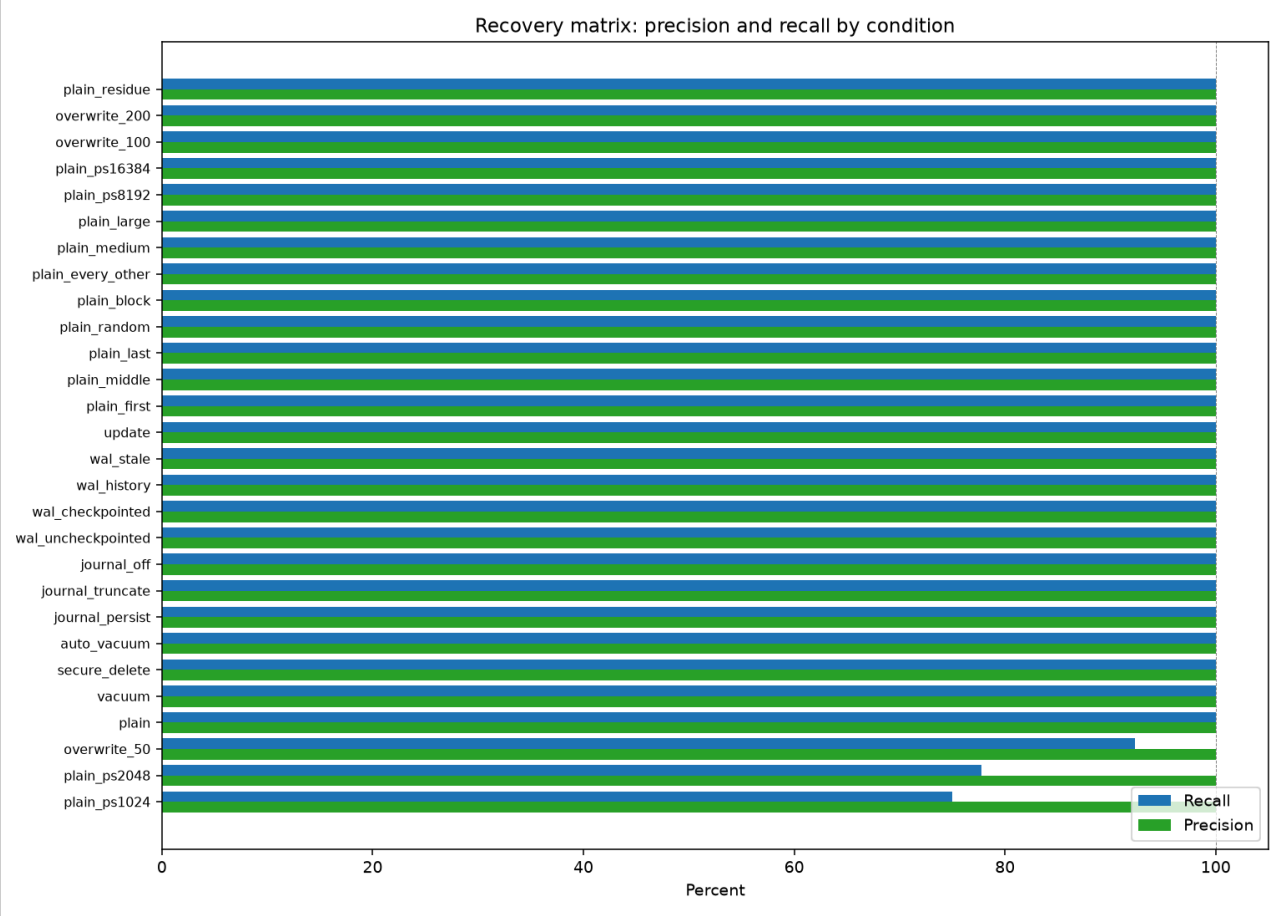


Figure 5. Recall by deletion condition. Three conditions fall short: plain 1024-byte pages (75%), plain 2048-byte pages (78%) and 50% overwrite (92%).

On confidence calibration, of the recovered artifacts that matched a genuinely deleted row, 29 rated HIGH and 64 rated MEDIUM were all correct against the real values (100% each); no LOW result appeared in that lab. This sample is small (93 artifacts over three seeds), so it shows the mechanism works as designed rather than a general precision curve.

4.4 Comparison with other recovery tools

Four tools were run on the same 28 cases. A tool is credited with a row when the row's unique token appears in its output outside the live data.

Tool	Recall of 511 rows
This tool	**507 (99.2%)**
DC3 SQLite Dissect 1.0.0	469 (91.8%)

undark (freespace mode)	223 (43.6%)
SQLite `.recover`	9 (1.8%); it targets corrupt databases, not deleted rows

The other tools lose most on WAL history (0 of 25 for each of them, against 25 of 25 here), rollback journals, UPDATE-superseded versions and randomly scattered deletions. This comparison has limits: the matrix was designed alongside this tool and so emphasises what it handles, only recall is comparable, and tools were run with default options.

4.5 Validation on a real acquisition

The tool was run on the Digital Corpora Android 9 (Pixel 3) image, a logical extraction of a decrypted `/data` partition with 28,977 files (7.4 GB). The container SHA-256 computed by the tool equals the hash the dataset author publishes.

Measure	Result
Databases found	605: 576 readable, 26 flagged encrypted-suspect, 3 not SQLite
Active extraction fidelity	11 of 11 tables and 243 of 243 rows match SQLite's own row counts
Artifacts	376 active; 21,129 recovered; 249,372 carved candidates (99.5% LOW, none HIGH)
Validation	3,703 checks: 2,407 valid, 30 partial, 1,261 unverified, 5 invalid (genuine anomalies in the image)
Messages against the author's own list	30 of 30 live SMS found; 2 of the 4 flagged-deleted SMS recovered (both from a WAL), 2 not found; no message was invented
Google Messages parser (added afterwards)	184 messages; direction agrees on 30 of 30 body-matched reference messages
Run time	546 s single-process (2.6 GB peak); recovery 181 s to 45 s and carving 166 s to 44 s with 8 workers, identical results

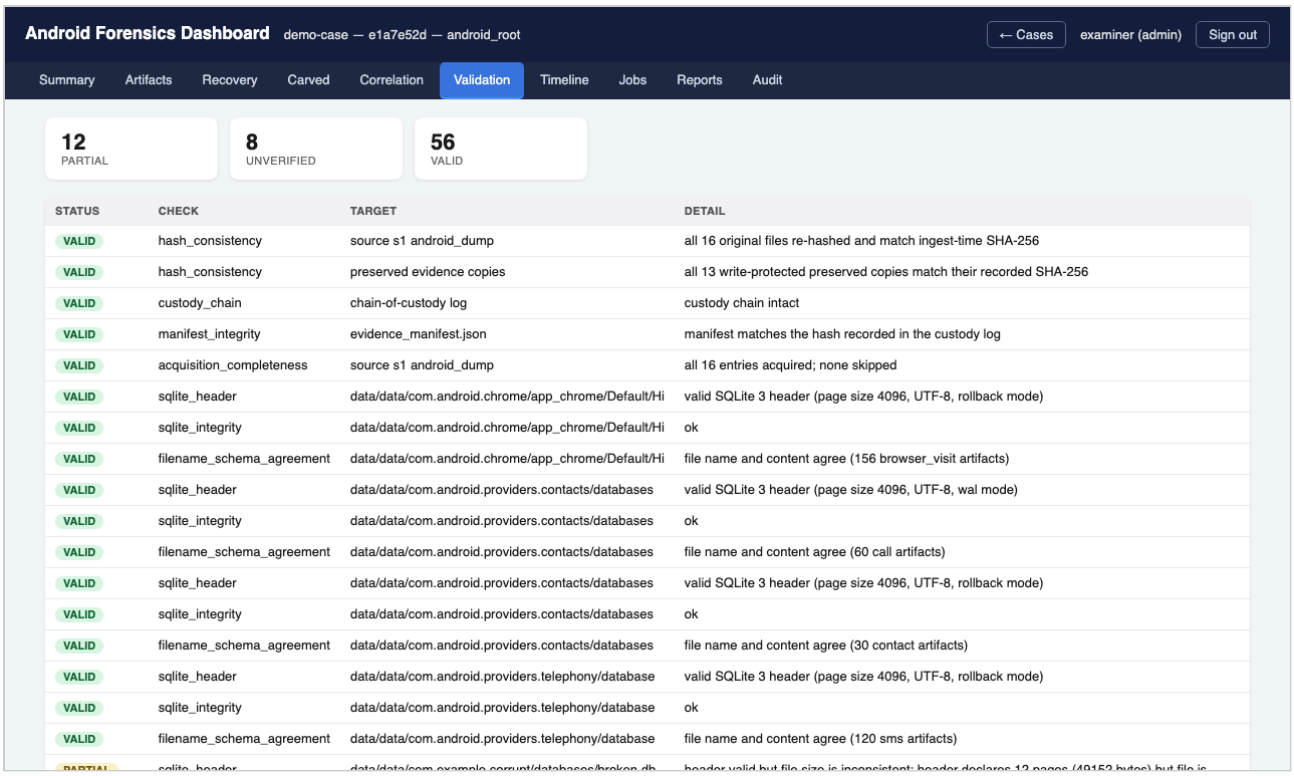


Figure 6. Validation results view.

> **Precision caveat.** Only SMS could be scored against an independent list. For everything else on real data there is no ground truth, so false-positive rates on real devices are unknown and LOW-confidence results should be treated as leads, not findings.

4.6 Test results

The suite contains 1,664 tests; all pass on the development machine and 94.1% of source lines are covered. Continuous integration ran the suite on Linux, macOS and Windows with Python 3.11, 3.12 and 3.13 and passed (the last full run was before the final desktop-application and typing changes), together with lint, type check, security scan, dependency audit, browser end-to-end tests, and a hardened Docker smoke test. The test areas are:

Area	Focus
Unit	SQLite pages, records, DDL, recovery (including an equivalence oracle against the original algorithm), WAL, journal, freelist, carving, correlation, scoring, storage, ext4/f2fs/E01 readers, custody and signing.
Integration	Command line, dashboard, report formats, synthetic lab end to end, deletion matrix, parallel-equals-serial, server roles and SSO, desktop application.
Security	Hostile archives and evidence, fuzzing of parsers and of the server, injection defences, sandbox limits, tamper detection.
End to end	Playwright browser test of the dashboard.

4.7 Performance

Measured on an Apple M-series Mac; each run is the whole pipeline in a fresh process, with peak memory read from the kernel.

Database rows	Wall time	Peak memory
10,000	3.8 s	131 MB
100,000	38 s	662 MB
300,000	127 s	1.7 GB
1,000,000	558 s	4.7 GB

Time is close to linear at 0.4 to 0.6 ms per row for the whole pipeline. A typical phone's messages, calls, contacts and browser databases are well under 100,000 rows, which is about a minute. These figures were measured before the parallel workers and the lighter evidence graph were added, since then the evidence graph uses less memory and recovery and carving can run in parallel.

4.8 Defects found and fixed through real-data testing

- Recovery re-parsed each cell header once per table, so 605 real databases did not finish in 35 minutes; now once per offset, 7 to 10 times faster with identical results.
- The encryption detector flagged hash-named cache directories on a decrypted image; now restricted to block-aligned lengths under policy roots.
- Correlation loaded a full model per artifact: 1.58 GB for a 13 MB database; a slim subject cut that to 0.66 GB, and tuple-backed edges to 467 MB at 100,000 rows.
- The carver scanned every timestamp for every phone hit (quadratic); now sorted spans with bisection.
- Duplicate merging was quadratic and stalled a Windows CI run; now indexed, with per-record commit batching in reconstruction.
- Fuzzing found an index error in the CREATE TABLE parser on malformed input; now a declared error.
- Several tests assumed one SQLite version or platform; they were made portable after runs on the other systems.

5. Conclusion

The project meets its objective. An Android acquisition is ingested read-only and integrity-checked, its databases are discovered and extracted, and deleted and superseded records are recovered from freed pages, freeblocks, the WAL and journals by parsing the file format directly. Findings are carved, correlated across independent sources and given a deterministic confidence level with written reasons, then validated and reported with a hash-chained audit trail and signed reports. Against physical ground truth it recovered 507 of 511 rows with no false positives, outperforming three reference tools on the same cases, and on a real Android 9 image it matched SQLite's own row counts exactly and recovered 2 of 4 messages the image author had marked deleted without inventing any.

5.1 Limitations

- **Encrypted storage.** File-based encryption without the key yields no plaintext. Encrypted data is detected and reported, never decrypted, and no encrypted partition has been tested.
- **One real device.** Real validation covers one Pixel 3 on Android 9. Other Android versions and Samsung, Xiaomi and other vendor schemas are untested.
- **Unknown real-world precision.** Outside SMS there is no ground truth on real data; LOW results are leads, not findings. Confidence calibration was measured on a small synthetic sample.
- **Coverage gaps.** Deleted files at file-system level are not recovered, only deleted SQLite records. WhatsApp, Signal and other app-specific parsers do not exist (they are handled generically and through a plugin API); MMS attachments and ABX XML are not decoded; ext4 inline data and f2fs compression are unsupported. Recovery does not attempt tables with fewer than three stored columns, WITHOUT ROWID or virtual tables.
- **Genuine overwriting.** Data overwritten by later writes, a vacuumed database or `secure_delete` zero-fill is gone; flash TRIM and garbage collection further limit recovery. The absence of a result is never evidence that data did not exist, and every report says so.
- **Acquisition.** The tool examines acquisitions; it does not acquire from a live phone.
- **Status.** This is an academic project. It has had no third-party security review or accreditation and does not claim court admissibility. The server's job sandbox is POSIX-only.

5.2 Future work

- Validate on more authorised real acquisitions across Android versions and vendors, including a genuinely encrypted image, and measure real precision.
- App-specific parsers for WhatsApp, Signal and Telegram; MMS and ABX decoding.
- File-level deleted-file recovery where the file system retains it; f2fs compression.
- Streaming WAL processing, a disk-backed evidence graph and a shared job queue for very large cases.
- Calibrating confidence against labelled real data; independent security and standards review.
- A standalone packaged desktop build for each operating system.

5.3 Ethical and legal considerations

- Use only on evidence the investigator is legally authorised to examine.
- The tool never executes recovered content and makes no network calls during analysis.

- Recovered content is sensitive; case folders must be treated as confidential.
- Hashes, the audit log and signatures support, but do not replace, the investigator's own chain-of-custody procedures.
- All synthetic data is invented. The public Digital Corpora image contains synthetic test accounts supplied by its author, and their credentials are not reproduced.

6. Inference

Each inference states what the evidence supports and where it stops.

Observation	Inference
The 28-case matrix recovered 507 of 511 rows with 0 false positives; the other tools found 9 to 469.	Reading pages, WAL frames and journals directly finds much that tools working from the live schema or free space alone miss, especially WAL history. The matrix was designed with the tool, so this shows relative behaviour on these conditions, not general superiority.
Recall fell to 75% and 78% on 1,024- and 2,048-byte pages and to 92% under 50% overwrite.	Recovery is bounded by what physically survives and by layout; the tool reports what it can prove rather than guessing the rest.
On the real image, active extraction matched SQLite's own counts for 243 of 243 rows, and 2 of 4 reference-deleted messages were recovered with none invented.	The extraction logic holds up on messy real data and the recovery does not fabricate. Two of four is a small sample and the misses were not analysed beyond the missing parser.
The real image yielded 249,372 carved candidates, 99.5% rated LOW and none HIGH.	Carving on caches is noisy by nature. Keeping candidates permanently below confirmed evidence, and filterable by confidence, keeps them from being mistaken for findings.
Running recovery on 605 real databases first took more than 35 minutes and was fixed to minutes with identical results.	Validation on synthetic data overstates quality: real data exposed performance and false-positive defects no synthetic case did. Real-data runs are essential.
8 workers made recovery about 4.0 times and carving about 3.8 times faster, with identical case content.	The per-database work is independent enough to parallelise safely; stages that remain single-process limit the whole-pipeline gain.
Of 8,244 recovered records in the synthetic run, only 26 were PARTIAL and the rest were superseded page versions.	Page-level recovery reports every provable earlier version, not only user deletions. A reviewer must read status and source, not just counts.
Six detectors and parsers (header cost, FBE heuristic, memory, carver, merge, DDL) were fixed only after real or fuzzed input.	Hostile-input testing and real corpora find defects that hand-written cases do not; both belong in the standard process.

Limits of these inferences. The synthetic results show the logic works as designed, not how often it succeeds on real devices. Real validation covers one device and one Android version, and performance figures describe one Mac.

7. Reference

- SQLite documentation: Database File Format; Write-Ahead Logging; Rollback Journal, https://www.sqlite.org/fileformat2.html.
- J. Hickman, Digital Corpora: Android 9 (Pixel 3) image, https://digitalcorpora.org/corpora/cell-phones/android-9/.
- DC3, SQLite Dissect, https://github.com/Defense-Cyber-Crime-Center/sqlite-dissect.
- undark, SQLite deleted-record recovery tool, https://github.com/inflex/undark.
- Linux kernel documentation: ext4 disk layout; F2FS, https://www.kernel.org/doc/html/latest/filesystems/.
- Expert Witness Compression Format (EWF) specification, libyal/libewf.
- NIST SP 800-101 Rev. 1, Guidelines on Mobile Device Forensics, 2014.
- NIST SP 800-86, Guide to Integrating Forensic Techniques into Incident Response, 2006.
- RFC 8032, Edwards-Curve Digital Signature Algorithm (Ed25519), IETF, 2017.
- Android Open Source Project, File-Based Encryption, https://source.android.com/docs/security/features/encryption/file-based.
- FastAPI, pydantic, Typer, PySide6 and pytest documentation.
- Project source, README and documentation (this submission), https://github.com/MithranHarsha/Android-Forensic-Analysis-Toolkit.

Appendix A. Running the tool and the demo

Task	Command
Install	`python3 -m venv .venv && .venv/bin/pip install -e ".[dev,gui]"`
Generate demo evidence with ground truth	`android-forensics synth synthetic_evidence`
Full analysis in one command	`android-forensics analyze <evidence> --cases-dir cases --name demo --workers auto`
Step by step	`init-case`, `ingest`, `scan`, `analyze`, `validate`, `report --format html,json,csv,pdf`
Desktop application	`android-forensics-gui`, or on macOS build the app with `packaging/macos/build_app.sh`
Read-only dashboard	`android-forensics dashboard demo` (http://127.0.0.1:8765)
Multi-user server	`android-forensics serve` (token authentication, roles, optional SSO)
Run the tests	`pytest`; with coverage: `coverage run -m pytest && coverage report`
Docker	The image runs as a non-root user with a read-only evidence mount; see `docs/deployment.md`

For a demonstration, run the synthetic generator, analyse the generated `acquisition.zip` by dropping it on the desktop application, and open the Recovered tab.

Appendix B. Case directory and database schema

Each case is a directory: `case.db`, `evidence/` (write-protected working copies), `reports/`, `scratch/`, `evidence_manifest.json` with its `.sha256`, and the external hash-chained `audit.log.jsonl`.

Table	Contents
cases, case_details, analysis_runs	Case identity, certification details, one row per pipeline stage run with parameters and statistics.
evidence_sources, evidence_files	Each source and file with SHA-256, size, times, kind and preserved-copy path.
databases, db_tables	Every SQLite database with header, version, sidecars, open and integrity status, and its tables, columns and schema variant.
artifacts	Active and reconstructed artifacts with provenance, confidence factors, timestamps and raw bytes.
recovered_records, wal_frames, freelist_pages, unrecoverable_regions	Recovered rows with page, offset, source and fit; WAL frame status; freelist structure; proven-empty regions.
carved_candidates	Carved matches with kind, offset, context and confidence.
correlations	Evidence-graph edges with the evidence for each.
validation_results, timeline_events, scan_issues	Check results, the unified timeline, and flagged filesystem entries.
custody_log, schema_migrations	Hash-chained in-database custody log and applied migrations.